MANUAL AVANZADO DE GIT Y ADMINISTRACIÓN EN GITHUB

De la Terminal a la Gobernanza Empresarial: Ramas Protegidas, Pull Requests y Resolución de Conflictos

Manual metodológico integral para ingenieros de software, administradores de plataformas y líderes técnicos. Diseñado bajo los estándares rigurosos de Onetel C.A. y enriquecido con la pedagogía técnica de freeCodeCamp. Cubre desde los fundamentos internos de Git (DAG, SHA-1, árboles y blobs), pasando por el ciclo completo de tres estados, estrategias de ramificación profesional, resolución quirúrgica de conflictos de código, hasta la administración avanzada de GitHub: gestión de organizaciones, control de acceso por roles (RBAC), reglas de protección de ramas (*Rulesets*), auditoría de seguridad y el flujo completo de revisión y aprobación de Pull Requests.

ÍNDICE GENERAL Y MAPA CURRICULAR

TABLA DE CONTENIDOS

Sección	Módulo / Tema Principal	Página	Tipo
Módulo 1	Arquitectura de Control de Versiones: Git vs. GitHub, DVCS y Configuración Global	Pág. 2	Teoría / CLI
Módulo 2	Las Tres Áreas de Git: Working Directory, Staging Area e Historial Sellado	Pág. 3	Flujo Local
Módulo 3	Inspección de Cambios (Log, Diff) y Técnicas Seguras de Deshacer (Restore, Stash)	Pág. 4	Auditoría
Módulo 4	Estrategias de Ramificación: GitFlow, Trunk-Based, Feature Branches y Fusión	Pág. 5	Branching
Módulo 5	Resolución Quirúrgica de Conflictos de Fusión (Merge Conflicts y Marcadores)	Pág. 6	Conflictos
Módulo 6	Integración Remota con GitHub: Criptografía SSH Ed25519, Push, Pull y Fetch	Pág. 7	Cloud Git
Módulo 7	Administración de GitHub: Organizaciones, Equipos (Teams), Visibilidad y RBAC	Pág. 8	Gobernanza
Módulo 8	Gobernanza de Código: Reglas de Protección de Ramas, Rulesets y CODEOWNERS	Pág. 9	Branch Rules
Módulo 9	Ciclo de Vida del Pull Request: Creación, Code Review, Aprobación y Merges	Págs. 10–11	Colaboración
Módulo 10	Estándar Conventional Commits, Buenas Prácticas y Plantilla .gitignore	Pág. 12	Estándares
Prácticas	Laboratorios Guiados 1, 2 y 3: Ciclo Completo, Conflictos Reales y Blindaje PR	Págs. 13–15	Hands-on Labs
Anexo	Hoja de Referencia Rápida (Cheat Sheet Profesional) y Matriz de Auditoría	Pág. 16	Cheat Sheet

Nombre del Participante:

Empresa / Departamento:

Usuario de GitHub / ID:

Instructor Responsable:

1.1 ¿QUÉ ES GIT Y QUÉ ES GITHUB? DIFERENCIAS Y MODELO DISTRIBUIDO

FUNDAMENTOS

Control de Versiones Centralizado vs. Distribuido (DVCS)

Evolución de los Sistemas de Control de Versiones: Antes de Git existían sistemas centralizados como CVS o Subversion (SVN), donde el historial residía en un único servidor. Si el servidor caía o no había conexión a internet, los ingenieros no podían crear puntos de control ni revisar diferencias de código.
Git (creado por Linus Torvalds en 2005) introdujo la arquitectura *Distribuida (DVCS)*: cada clon local es un repositorio completo con el historial íntegro de confirmaciones. El trabajo local es instantáneo y autónomo.
GitHub es la plataforma SaaS líder mundial que aloja repositorios Git en la nube, añadiendo capas de colaboración empresarial: auditoría mediante Pull Requests, gestión de accesos, integración continua (CI/CD) y análisis automático de vulnerabilidades.

Dimensión	Git (Motor Local CLI)	GitHub (Plataforma Colaborativa Cloud)
Naturaleza	Herramienta de línea de comandos open-source instalada en el sistema.	Servicio web alojado en la nube con interfaces visuales y API REST/GraphQL.
Conectividad	Opera 100% desconectado: commits, ramas, historial y diffs no requieren internet.	Requiere red para sincronizar repositorios, auditar PRs o disparar pipelines.
Responsabilidad	Rastrear cambios en archivos, calcular hashes SHA-1 y gestionar árboles.	Colaboración en equipo, permisos RBAC, gobernanza de código e Issues.
Almacenamiento	Directorio oculto local `.git/` en el disco de tu máquina.	Infraestructura global distribuida con alta disponibilidad y respaldos.

1.2 Configuración Global de Identidad y Entorno

BASH: CONFIGURACIÓN GLOBAL INICIAL EN TERMINAL — Terminal

```
# 1. Verificar versión instalada de Git
git --version

# 2. Configurar firma de autoría (debe coincidir exactamente con tu cuenta de GitHub)
git config --global user.name "Ing. Carlos Mendoza"
git config --global user.email "cmendoza@onetel.cloud"

# 3. Establecer 'main' como nombre predeterminado de la rama inicial
git config --global init.defaultBranch main

# 4. Configurar el editor predeterminado para mensajes de commit interactivos (VS Code)
git config --global core.editor "code --wait"

# 5. Normalizar terminaciones de línea (CRLF en Windows vs LF en Linux/macOS)
git config --global core.autocrlf input

# 6. Listar y comprobar la configuración activa
git config --list --show-origin
```

Principio de Autoría Inmutable: Una vez creado un commit, el nombre y el correo configurados quedan sellados en el hash SHA-1 de esa confirmación. Configurar la identidad correcta desde el día 1 evita tener que reescribir historiales posteriormente.

2.1 EL CICLO DE VIDA DE UN ARCHIVO Y LAS TRES ÁREAS DE GIT

FLUJO LOCAL

Modelo Mental: Working Directory, Staging Area y Repositorio

Las Tres Zonas Lógicas de Git: A diferencia de otros sistemas que guardan cada cambio de archivo inmediatamente, Git implementa un buffer intermedio de precisión:
• **1. Working Directory (Directorio de Trabajo):** Archivos físicos en tu disco duro que estás editando en tu editor de código.
• **2. Staging Area (Índice o Prep):** Zona de empaquetado intermedio en memoria. Permite seleccionar quirúrgicamente qué líneas o archivos formarán parte de tu próximo commit.
• **3. Repositorio Local (Head / `.git/`):** La base de datos permanente donde se sellan los commits con sus metadatos y punteros inmutables.

DIAGRAMA: TRANSICIONES DE ESTADO EN EL FLUJO LOCAL — Arquitectura Interna



2.2 Inicialización, Estados de Archivo y Primer Commit

BASH: CICLO COMPLETO DE CONFIRMACIÓN LOCAL — Terminal

```bash
# 1. Crear directorio del proyecto e ingresar a él
mkdir proyecto-onetel-portal && cd proyecto-onetel-portal

# 2. Inicializar el repositorio Git (crea el subdirectorio oculto .git/)
git init

# 3. Crear archivos de código fuente y documentación
echo "# Onetel Portal API" > README.md
echo "<!DOCTYPE html><html><body><h1>Onetel</h1></body></html>" > index.html

# 4. Inspeccionar el estado de los archivos (Aparecerán en rojo: Untracked)
git status

# 5. Agregar archivos de forma selectiva al Staging Area
git add README.md           # Agrega un archivo específico
git add index.html          # Agrega otro archivo específico
# git add .                 # Alternativa: agrega todas las modificaciones del directorio

# 6. Comprobar que los archivos están preparados (Aparecerán en verde: Changes to be committed)
git status -s

# 7. Crear el Commit sellado con mensaje descriptivo bajo estándar profesional
git commit -m "feat: inicializar estructura de proyecto con README e index base"
```

Analogía Didáctica: El *Working Directory* es tu mesa de carpintería donde tienes herramientas y piezas a medio construir. El *Staging Area* es la caja de encomienda donde decides meter únicamente las piezas que están 100% terminadas y listas para entrega. El *Commit* es el sellado hermético de la caja con precinto de seguridad, fechada y firmada.

3.1 AUDITORÍA CON GIT LOG, DIFF Y TÉCNICAS SEGURAS DE DESHACER

HISTORIAL Y DESHACER

Inspección Profunda: Log y Diff

Viajar en el Tiempo con Certeza Criptográfica: Cada confirmación genera un hash SHA-1 inmutable (ej. `7f3a8b2...`). Con `git log` auditamos quién, cuándo y por qué se introdujo cada cambio. Mediante `git diff` visualizamos las diferencias exactas línea por línea antes de preparar o confirmar código, evitando subir código no deseado.

BASH: COMANDOS DE INSPECCIÓN Y COMPARACIÓN AVANZADA — Terminal

```
# 1. Historial compacto con grafo visual de bifurcaciones y fusiones
git log --oneline --graph --all --decorate

# 2. Filtrar historial por autor o por rango de fechas
git log --author="Carlos" --since="2 weeks ago"

# 3. Ver qué archivos cambiaron y cuántas líneas se insertaron/borraron
git log --stat -n 3

# 4. Comparar el Working Directory frente al Staging Area (qué cambié sin preparar)
git diff

# 5. Comparar el Staging Area frente al último commit (qué estoy a punto de commitear)
git diff --staged

# 6. Inspeccionar un commit específico por su hash
git show 7f3a8b2
```

3.2 Protocolos para Deshacer Cambios sin Romper el Proyecto

BASH: DESHACER CAMBIOS SEGÚN EL ÁREA DONDE OCURRIERON — Terminal

```
# ESCENARIO A: Descartar cambios en disco de un archivo aún no preparado (No en Staging)
git restore index.html

# ESCENARIO B: Retirar un archivo del Staging Area preservando los cambios en disco
git restore --staged index.html

# ESCENARIO C: Modificar el mensaje del último commit o añadirle archivos olvidados
git add archivo_olvidado.txt
git commit --amend -m "fix: agregar archivo_olvidado y corregir mensaje"

# ESCENARIO D: Revertir un commit público creando un nuevo commit inverso (SEGURO PARA EQUIPOS)
git revert 7f3a8b2

# ESCENARIO E: Guardar cambios a medio terminar en gaveta temporal sin hacer commit (Stash)
git stash                    # Almacena en la pila temporal y limpia el Working Directory
git stash list               # Lista los stashes guardados
git stash pop                # Restaura los cambios y elimina la entrada de la pila
```

Regla de Oro en Equipos de Software: Jamás uses `git reset --hard` en ramas remotas o compartidas. `reset` reescribe la historia y destruye commits. Para anular cambios en código compartido, utiliza siempre `git revert`, el cual genera un commit nuevo que desace con elegancia las modificaciones previas sin romper a tus compañeros.

4.1 RAMAS (BRANCHES), WORKFLOWS Y FUSIÓN DE HISTORIAL

BRANCHING STRATEGIES

¿Qué es una Rama en Git?

Un Puntero Móvil Ultraligero: En Git, una rama no es una copia pesada de toda la carpeta del proyecto. Es simplemente un archivo de 41 bytes que contiene el hash SHA-1 del commit más reciente de esa línea de desarrollo. Al crear una rama, creas una línea de tiempo paralela para construir funcionalidades (*features*) o corregir errores (*hotfixes*) sin desestabilizar la rama de producción `main`.

Modelos de Trabajo: GitFlow vs. Trunk-Based Development

• **GitFlow:** Modelo formal para software empaquetado o empresarial con ciclos de lanzamiento programados. Utiliza dos ramas infinitas (`main` para código en producción y `develop` para integración continua), más ramas de soporte: `feature/*`, `release/*` y `hotfix/*`.
• **Trunk-Based / Feature Branching (Recomendado en Onetel):** Modelo ágil para entrega continua. Se mantiene una rama `main` siempre desplegable; los desarrolladores abren ramas efímeras de vida corta (1 a 3 días, ej. `feat/autenticacion-jwt`) que se fusionan rápidamente a `main` mediante Pull Requests tras superar pruebas automatizadas.

BASH: CREACIÓN, NAVEGACIÓN Y FUSIÓN DE RAMAS — Terminal

```bash
# 1. Listar todas las ramas locales (la rama activa se marca con asterisco *)
git branch

# 2. Crear una nueva rama e ingresar a ella en un solo paso moderno
git switch -c feat/modulo-facturacion
# (Sintaxis clásica equivalente: git checkout -b feat/modulo-facturacion)

# 3. Desarrollar la funcionalidad y crear confirmaciones atómicas
echo "function facturar() {}" > facturacion.js
git add facturacion.js
git commit -m "feat: implementar motor de cálculo de facturas"

# 4. Regresar a la rama principal antes de fusionar
git switch main

# 5. Fusionar la rama de la funcionalidad en main (Merge)
git merge feat/modulo-facturacion

# 6. Eliminar la rama secundaria tras completarse la fusión
git branch -d feat/modulo-facturacion
```

Diferencia: Fast-Forward vs. Recursive Merge (3-Way): Si la rama `main` no avanzó mientras trabajabas en tu rama, Git simplemente mueve el puntero de `main` hacia adelante (*Fast-Forward*). Si `main` recibió otros commits mientras trabajabas, Git realiza un *3-Way Merge* creando un commit especial de fusión que combina ambos historiales.

5.1 RESOLUCIÓN DE CONFLICTOS DE FUSIÓN (MERGE CONFLICTS)

CONFLICT RESOLUTION

¿Por qué Ocurre un Conflicto de Fusión?

La Naturaleza del Conflicto: Git es extremadamente inteligente fusionando cambios en diferentes archivos o en líneas distintas del mismo archivo. Sin embargo, cuando dos ramas modifican **las mismas líneas exactas** de un archivo desde su ancestro común, Git se detiene deliberadamente. No intenta adivinar qué versión es la correcta; pausa el proceso de fusión y le pide al desarrollador que decida.

TEXT: ANATOMÍA DE LOS MARCADORES DE CONFLICTO — config.json (En estado de conflicto)

```
<<<<<<< HEAD (Rama receptora actual, ej: main)
  "puerto_servidor": 8080,
  "modo_depuracion": false
=======
  "puerto_servidor": 9000,
  "modo_depuracion": true
>>>>>>> feat/servidor-local (Rama entrante que se intenta fusionar)
```

5.2 Procedimiento Paso a Paso para Resolver un Conflicto

BASH: PROTOCOLO OPERATIVO DE RESOLUCIÓN EN TERMINAL — Terminal

```
# 1. Al ejecutar el merge, Git informa el conflicto y suspende la operación:
git merge feat/servidor-local
# Salida: CONFLICT (content): Merge conflict in config.json
#         Automatic merge failed; fix conflicts and then commit the result.

# 2. Identificar qué archivos específicos están en conflicto
git status

# 3. Abrir el archivo en VS Code o tu editor.
# Decidir qué código debe quedar y BORRAR COMPLETAMENTE las líneas con <<<<<<<, ======= y >>>>>>>

# 4. Una vez corregido y guardado el archivo, agregarlo a Staging para marcarlo como resuelto
git add config.json

# 5. Completar formalmente la fusión creando el commit de resolución
git commit -m "merge: resolver conflicto de puerto en config.json conservando puerto 8080"

# NOTA: Si en algún momento deseas abortar el merge y regresar todo al estado previo:
# git merge --abort
```

Buenas Prácticas en Prevención de Conflictos: Para minimizar conflictos en equipos grandes: (1) Crea ramas pequeñas de vida corta enfocadas en una sola tarea, (2) Ejecuta `git pull` con frecuencia para mantener tu rama local actualizada con `main`, y (3) Comunica a tu equipo cuando vayas a refactorizar archivos centrales de configuración.

6.1 CONEXIÓN REMOTA CON GITHUB: SSH, PUSH, PULL Y FETCH

GITHUB CLOUD

Autenticación Criptográfica Asimétrica (SSH Ed25519)

Eliminación de Contraseñas en Terminal: GitHub prohíbe el uso de contraseñas de cuenta para operaciones como `git push`. El estándar industrial obligatorio es el uso de llaves criptográficas **SSH (Secure Shell)** sobre la curva de Edwards **Ed25519**. Tu computador genera un par de llaves: la *llave privada* (secreta en tu disco) y la *llave pública* (que copias y registras en los ajustes de tu perfil de GitHub). Cada conexión se firma matemáticamente sin exponer credenciales.

BASH: GENERACIÓN E INSTALACIÓN DE LLAVE SSH ED25519 — Terminal

```bash
# 1. Generar el par de llaves con algoritmo Ed25519
ssh-keygen -t ed25519 -C "cmendoza@onetel.cloud" -f ~/.ssh/id_ed25519 -N ""

# 2. Configurar permisos POSIX estrictos en el directorio .ssh (Obligatorio en OpenSSH)
chmod 700 ~/.ssh && chmod 600 ~/.ssh/id_ed25519 && chmod 644 ~/.ssh/id_ed25519.pub

# 3. Iniciar el agente SSH y cargar la clave privada
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# 4. Imprimir la clave pública para copiarla en GitHub (Settings > SSH and GPG keys)
cat ~/.ssh/id_ed25519.pub

# 5. Comprobar la conexión con los servidores de GitHub
ssh -T git@github.com
# Salida esperada: Hi cmendoza! You've successfully authenticated...
```

6.2 Comandos de Sincronización Remota

BASH: VINCULACIÓN REMOTA, PUSH, PULL Y CLONACIÓN — Terminal

```bash
# 1. Vincular el repositorio local con un repositorio creado en GitHub
git remote add origin git@github.com:onetel-cloud/portal-core.git

# 2. Verificar los remotos vinculados (debe listar URLs para fetch y push)
git remote -v

# 3. Enviar la rama main por primera vez y vincular el rastreo (upstream -u)
git push -u origin main

# 4. Descargar y fusionar los cambios más recientes que otros compañeros subieron
git pull origin main

# 5. Descargar metadatos remotos SIN alterar los archivos locales de trabajo
git fetch origin

# 6. Clonar un repositorio existente desde GitHub a una nueva estación de trabajo
git clone git@github.com:onetel-cloud/portal-core.git
```

7.1 GOBERNANZA EN GITHUB: ORGANIZACIONES, EQUIPOS Y CONTROL RBAC

GITHUB ADMIN

Cuentas Personales vs. Organizaciones de GitHub (GitHub Orgs)

Estructura Empresarial: Las cuentas personales pertenecen a un individuo y no deben utilizarse para proyectos corporativos. En entornos profesionales se crean **Organizaciones** (ej. `github.com/onetel-cloud`), las cuales actúan como contenedores compartidos con facturación centralizada, políticas de seguridad globales y gestión estructurada de equipos mediante el principio de menor privilegio (*Role-Based Access Control - RBAC*).

Rol en Repositorio	Permisos y Alcance Técnico	Perfil Típico Asignado
Read	Clonar código, leer discusiones, reportar Issues y abrir Pull Requests.	Contratistas externos, auditores o pasantes.
Triage	Gestionar Issues, etiquetas y milestones (sin permisos de escritura en código).	Project Managers y personal de soporte.
Write	Hacer push directo a ramas no protegidas, crear ramas y gestionar PRs.	Desarrolladores e ingenieros de software activos.
Maintain	Administrar configuraciones del repositorio sin acceso a borrado o facturación.	Líderes técnicos y coordinadores de proyecto.
Admin	Control total: borrar repositorio, definir reglas de protección y transferir propiedad.	Arquitectos y administradores de plataforma.

7.2 Administración de Equipos (Teams) y Repositorios Privados

TEXT: GUÍA DE CONFIGURACIÓN DE ORGANIZACIÓN Y EQUIPOS EN GITHUB — Portal GitHub UI

```
1. Crear la Organización:
   Perfil > Your organizations > New organization > Plan Teams o Enterprise > Nombre: onetel-cloud

2. Estructuración Jerárquica de Equipos (Teams):
   Organización > Teams > New team:
   • @onetel-cloud/frontend-devs  (Permiso de escritura en repositorios de interfaz)
   • @onetel-cloud/backend-devs   (Permiso de escritura en APIs y microservicios)
   • @onetel-cloud/security-audit (Permiso de lectura en toda la organización)

3. Configuración de Repositorios Privados e Higiene de Acceso:
   • Repositorio > Settings > General > Danger Zone > Change visibility > "Make private".
   • Repositorio > Collaborators and teams > Add team: Asignar equipos en vez de usuarios individuales.
   • Política de 2FA Mandatoria: Org Settings > Authentication security > Activar "Require two-factor authentication
```

Principio de Mínimo Privilegio: Jamás asignes el rol `Admin` a un equipo de desarrollo completo. Si un desarrollador necesita permisos de escritura, asígnale el rol `Write` y protege la rama `main` con reglas de revisión para asegurar que nadie pueda romper producción accidentalmente.

8.1 REGLAS DE PROTECCIÓN DE RAMAS (BRANCH RULES) Y CODEOWNERS

BRANCH GOVERNANCE

¿Por qué Proteger la Rama Principal?

Bloqueo de Push Directo a Producción: En un entorno empresarial, permitir que cualquier desarrollador ejecute `git push origin main` es una falla crítica de seguridad y estabilidad. Las **Branch Protection Rules** (o Rulesets de GitHub) obligan a que todo cambio pase por una solicitud formal de revisión (Pull Request), supere pruebas automatizadas de integración continua (CI) y reciba la aprobación obligatoria de otros ingenieros.

8.2 Implementación del Archivo .github/CODEOWNERS

El archivo `.github/CODEOWNERS` permite definir automáticamente qué ingenieros o equipos son responsables de áreas específicas del código. Cuando un Pull Request modifica esos archivos, GitHub asigna automáticamente como revisores obligatorios a los propietarios correspondientes.

TEXT: ARCHIVO DE DUEÑOS DE CÓDIGO (.github/CODEOWNERS) — .github/CODEOWNERS

```
# Propietario por defecto para todo el repositorio
*                               @onetel-cloud/tech-leads

# El equipo de frontend debe aprobar cualquier cambio en vistas y CSS
/frontend/                      @onetel-cloud/frontend-team
*.css                           @onetel-cloud/frontend-team

# El equipo de seguridad y backend debe auditar la lógica de autenticación
/api-node/src/middlewares/auth.js   @onetel-cloud/security-team
/api-laravel/routes/api.php         @onetel-cloud/backend-team

# Cambios en infraestructura de despliegue y scripts
/scripts/                       @onetel-cloud/devops-team
```

8.3 Configuración de Branch Protection Rules en la Interfaz de GitHub

GUÍA: PASO A PASO PARA PROTEGER LA RAMA MAIN — GitHub UI Settings

```
1. En el repositorio de GitHub, navegar a: Settings > Branches > Add branch protection rule.
2. En "Branch name pattern", escribir: main (o patrón de producción: release/*).
3. Activar las siguientes casillas mandatorias:
   [✔] Require a pull request before merging:
       - Require approvals: Seleccionar al menos 1 o 2 aprobaciones de revisores.
       - Dismiss stale pull request approvals when new commits are pushed (Seguridad: si el autor añade nuevo código
       - Require review from Code Owners (Obliga aprobación según archivo CODEOWNERS).
   [✔] Require status checks to pass before merging:
       - Exige que los tests automatizados de CI/CD pasen exitosamente antes de habilitar el botón Merge.
   [✔] Do not allow bypassing the above settings (Aplica la regla incluso para administradores).
   [✔] Block force pushes (Impide 'git push --force' que destruiría el historial de producción).
   [✔] Block deletions (Impide que la rama principal sea eliminada por error).
```

9.1 PULL REQUESTS: CREACIÓN, REVISIÓN DE CÓDIGO Y AUDITORÍA

PULL REQUEST LIFECYCLE

¿Qué es un Pull Request (PR)?

La Herramienta Suprema de Colaboración: Un Pull Request no es un comando de Git, sino una funcionalidad exclusiva de plataformas como GitHub. Es una solicitud formal donde un ingeniero notifica a su equipo: *"He terminado esta funcionalidad en mi rama secundaria; solicito que revisen mis cambios, discutan mejoras y aprueben la fusión hacia la rama principal"*. Es el punto neurálgico del control de calidad en la ingeniería moderna.

Forking Workflow vs. Branching Workflow

• **Branching Workflow (Equipos Internos):** Todos los miembros tienen acceso de escritura al mismo repositorio empresarial. Crean ramas descriptivas directas (ej. `feat/telemetria`) y abren PRs internos hacia `main`.
• **Forking Workflow (Open Source o Contratistas):** El colaborador no tiene permisos de escritura en el repositorio oficial. Hace un *Fork* (copia espejo a su cuenta personal de GitHub), realiza sus cambios allí, y abre un Pull Request desde su cuenta personal hacia el repositorio oficial.

BASH & UI: FLUJO INTEGRAL DE CREACIÓN DE UN PULL REQUEST — Terminal y GitHub

```
# 1. Asegurarse de tener la última versión de main antes de empezar
git switch main && git pull origin main

# 2. Crear una rama de funcionalidad con nombre estandarizado
git switch -c feat/modulo-reportes-pdf

# 3. Desarrollar el código, probarlo localmente y hacer commits atómicos
git add src/reportes.js
git commit -m "feat: implementar generador de reportes en PDF con formato A4"

# 4. Subir la rama a GitHub
git push -u origin feat/modulo-reportes-pdf

# 5. En la interfaz web de GitHub aparecerá el botón: "Compare & pull request".
# Rellenar la plantilla con:
# - Título: feat: agregar exportación de reportes PDF
# - Descripción: Qué problema resuelve, pasos para probarlo y capturas de pantalla si aplica.
# - Asignar Reviewers (Revisores) y vincular el Issue correspondiente (ej: Closes #42).
```

Draft Pull Requests (Borradores): Si tu trabajo aún está en desarrollo pero deseas que tus compañeros vean la dirección técnica o consultar dudas sobre el código, puedes crear un *Draft Pull Request*. Esto deshabilita deliberadamente la opción de fusionar hasta que tú mismo lo marques formalmente como *Ready for review*.

9.2 CÓMO REVISAR, COMENTAR Y APROBAR UN PULL REQUEST

CODE REVIEW & MERGE

El Rol del Revisor de Código (Code Reviewer)

Cultura Constructiva de Calidad: La revisión de código no es una auditoría punitiva; es un proceso colaborativo para detectar bugs, asegurar apego a estándares de arquitectura, evitar código duplicado y compartir conocimiento. Un revisor inspecciona la pestaña *Files changed*, comenta líneas específicas y emite un veredicto formal.

GUÍA: LOS 3 ESTADOS DE VEREDICTO EN UN PULL REQUEST — GitHub UI Review Options

```
Al pulsar el botón verde "Review changes" en GitHub, el revisor elige entre 3 opciones:

1. [ Comment ]:
   Envía retroalimentación general o preguntas aclaratorias sin aprobar ni rechazar el PR.
   Ideal para solicitar clarificaciones de requerimientos o compartir sugerencias no bloqueantes.

2. [ Request changes ]:
   Bloquea la fusión del PR hasta que el autor resuelva observaciones críticas (ej. fallas de seguridad,
   carencia de pruebas unitarias o nombres erróneos). Requiere justificar detalladamente qué debe modificarse.

3. [ Approve ]:
   Otorga el visto bueno formal. Confirma que el código cumple con los estándares técnicos y está listo
   para producción. Si la rama exige 1 o 2 aprobaciones, este estado contabiliza para habilitar el botón Merge.
```

9.3 Las 3 Estrategias de Fusión en GitHub (Merge Types)

Estrategia	Comportamiento en el Historial	Cuándo Usarlo (Criterio Onetel)
Create a merge commit	Conserva todos los commits individuales de la rama y añade un commit explícito de fusión (Merge commit).	Ramas grandes donde se requiere auditar cronológicamente cada paso intermedio.
Squash and merge *(Estándar Recomendado)*	Aplasta todos los commits de la rama secundaria en **un único commit limpio** sobre `main`.	Pull Requests de funcionalidades puntuales. Evita ensuciar `main` con commits intermedios (ej: "fix typo").
Rebase and merge	Mueve los commits de la rama secundaria al final de `main` de forma lineal sin crear commit de merge.	Equipos que exigen un historial estrictamente lineal sin bifurcaciones en el grafo.

9.4 Auditoría y Revisión Local con GitHub CLI (gh)

BASH: REVISIÓN DE PULL REQUESTS DESDE LA TERMINAL — GitHub CLI Workflow

```bash
# 1. Listar los Pull Requests abiertos en el repositorio
gh pr list --state open

# 2. Descargar y cambiar a la rama exacta del PR para probarlo localmente
gh pr checkout 42

# 3. Inspeccionar las diferencias completas introducidas por el PR
gh pr diff

# 4. Ejecutar pruebas automatizadas en tu entorno local antes de dar visto bueno
npm test || pytest

# 5. Aprobar formalmente el PR desde la línea de comandos con comentario
gh pr review 42 --approve -b "LGTM: Pruebas locales ejecutadas y superadas sin incidencias."
```

Checklist Pre-Fusión Onetel: Antes de presionar *Squash and merge* asegúrate de que: (1) Todos los hilos de conversación estén marcados como *Resolved*, (2) Los checks automáticos de CI/CD estén en verde, (3) La rama esté al día con `main` y (4) La descripción sintetice claramente el valor técnico entregado.

10.1 ESTÁNDAR CONVENTIONAL COMMITS Y GESTIÓN DE .GITIGNORE

BEST PRACTICES

Estándar de Commits Semánticos (Conventional Commits 1.0)

Estructura Estandarizada: Los mensajes genéricos como `"arreglos"` o `"cambios varios"` impiden rastrear bugs o generar notas de versión (*changelogs*). El estándar internacional prescribe la estructura: `tipo(alcance_opcional): descripción breve en imperativo`.

Prefijo	Significado y Propósito	Ejemplo Estandarizado
`feat:`	Introduce una nueva característica o funcionalidad.	`feat(auth): agregar soporte para login vía OAuth2`
`fix:`	Corrige un defecto, bug o vulnerabilidad en el sistema.	`fix(api): corregir cálculo de timeout en cliente HTTP`
`docs:`	Cambios exclusivos en documentación técnica o README.	`docs: actualizar diagrama de arquitectura en README`
`refactor:`	Reestructuración interna de código sin alterar su comportamiento.	`refactor: simplificar bucles en motor de telemetría`
`test:`	Añade o corrige pruebas unitarias o de integración.	`test(login): agregar pruebas de carga con Mock`
`chore:`	Tareas rutinarias de mantenimiento, dependencias o tooling.	`chore: actualizar versión de Node.js en workflow`

10.2 Configuración Exhaustiva de .gitignore

TEXT: PLANTILLA PROFESIONAL DE ARCHIVOS EXCLUIDOS — .gitignore

```
# Dependencias externas (pesadas y reproducibles por gestores de paquetes)
node_modules/
vendor/
.venv/
__pycache__/

# Archivos de entorno y credenciales (SEGURIDAD CRÍTICA: JAMÁS SUBIR AL REPO)
.env
.env.*
!.env.example

# Archivos temporales, logs y volcados de depuración
*.log
npm-debug.log*
yarn-debug.log*

# Archivos de configuración de sistemas operativos
.DS_Store
Thumbs.db

# Carpetas de compilación y artefactos de distribución
dist/
build/
coverage/

# Bases de datos locales de prueba
database/*.sqlite
```

EJERCICIO PRÁCTICO GUIADO 1: DE LA TERMINAL LOCAL A GITHUB

HANDS-ON LAB 1

Objetivo del Laboratorio: Construir un repositorio desde cero, crear un flujo de tres confirmaciones semánticas, vincular llaves SSH y sincronizarlo con un repositorio privado en GitHub.

BASH: PASO A PASO DEL LABORATORIO 1 — Terminal Linux / macOS / WSL2

```
# PASO 1: Crear el espacio de trabajo local e iniciar Git
mkdir lab1-git-onetel && cd lab1-git-onetel
git init

# PASO 2: Configurar exclusiones antes de crear archivos
cat > .gitignore << 'EOF'
.env
*.log
node_modules/
EOF

# PASO 3: Crear el archivo README y la plantilla de configuración de variables
cat > README.md << 'EOF'
# Portal de Telemetría Onetel
Sistema de monitoreo para servidores industriales.
EOF

cat > .env.example << 'EOF'
PORT=8000
DATABASE_URL=sqlite:///db.sqlite
EOF

# PASO 4: Preparar los archivos y crear el commit inicial
git add .
git commit -m "chore: inicializar estructura de proyecto y reglas de exclusión"

# PASO 5: Crear el código de la aplicación y segundo commit
cat > server.js << 'EOF'
console.log("Servidor Onetel Telemetry Core iniciado.");
EOF

git add server.js
git commit -m "feat: implementar script de entrada del servidor de telemetría"

# PASO 6: Auditar el historial resultante
git log --oneline

# PASO 7: Ir a GitHub, crear el repositorio privado 'lab1-git-onetel' y vincularlo
git remote add origin git@github.com:tu-usuario/lab1-git-onetel.git
git push -u origin main
```

Criterio de Evaluación: Ingresa al navegador y verifica que en GitHub aparezcan los dos commits, el archivo `README.md` renderizado en la página principal y que el archivo `.env` (si lo creas localmente con datos de prueba) no aparezca en el repositorio remoto.

EJERCICIO PRÁCTICO GUIADO 2: SIMULACIÓN DE CONFLICTO Y MERGE

HANDS-ON LAB 2

Objetivo del Laboratorio: Provocar de forma deliberada un conflicto de fusión entre dos ramas que modifican la misma línea en un archivo y resolverlo quirúrgicamente en la terminal.

BASH: SIMULACIÓN CONTROLADA Y RESOLUCIÓN DE CONFLICTO — Terminal

```
# PASO 1: Asegurarse de estar en la rama main limpia
git switch main

# PASO 2: Crear un archivo base versionado
echo "VERSION_SISTEMA=1.0.0" > version.txt
git add version.txt && git commit -m "chore: definir version base 1.0.0"

# PASO 3: Crear rama secundaria 'feat/actualizar-v2' y modificar la línea
git switch -c feat/actualizar-v2
echo "VERSION_SISTEMA=2.0.0-PROD" > version.txt
git add version.txt && git commit -m "feat: subir version a 2.0.0 para produccion"

# PASO 4: Regresar a main y simular que otro desarrollador modificó la misma línea
git switch main
echo "VERSION_SISTEMA=1.1.0-HOTFIX" > version.txt
git add version.txt && git commit -m "fix: parche critico a version 1.1.0"

# PASO 5: Intentar fusionar la rama secundaria en main (SE DESATARÁ EL CONFLICTO)
git merge feat/actualizar-v2
# Salida: Auto-merging version.txt
#         CONFLICT (content): Merge conflict in version.txt
#         Automatic merge failed; fix conflicts and then commit the result.

# PASO 6: Inspeccionar el archivo con marcadores de conflicto
cat version.txt
# <<<<<<< HEAD
# VERSION_SISTEMA=1.1.0-HOTFIX
# =======
# VERSION_SISTEMA=2.0.0-PROD
# >>>>>>> feat/actualizar-v2

# PASO 7: Resolver el conflicto dejando el valor unificado correcto
echo "VERSION_SISTEMA=2.0.0-HOTFIX-CONSOLIDADO" > version.txt

# PASO 8: Agregar y completar el commit de fusión
git add version.txt
git commit -m "merge: resolver conflicto de versiones integrando hotfix en v2.0.0"

# PASO 9: Verificar que el grafo quedó unificado limpiamente
git log --oneline --graph -n 4
```

Resultado Exitoso: Al ejecutar `git status` la terminal debe responder *"nothing to commit, working tree clean"* y el archivo `version.txt` debe contener exactamente la cadena limpia sin caracteres residuales `<<<<<<<` ni `=======`.

EJERCICIO PRÁCTICO GUIADO 3: PULL REQUEST, BRANCH RULES Y CODEOWNERS

HANDS-ON LAB 3

Objetivo del Laboratorio: Configurar reglas de protección en GitHub para blindar la rama `main`, añadir dueños de código (CODEOWNERS), abrir un Pull Request desde una rama secundaria y simular la aprobación formal de un revisor.

FLUJO COMPLETO: DEFENSA DE CÓDIGO Y CODE REVIEW — Terminal y GitHub Web UI

```
# PASO 1: En tu repositorio local, crear la carpeta .github y el archivo CODEOWNERS
mkdir -p .github
cat > .github/CODEOWNERS << 'EOF'
* @tu-usuario
/seguridad/ @tu-usuario
EOF

git add .github/CODEOWNERS
git commit -m "chore: configurar archivo CODEOWNERS"
git push origin main

# PASO 2: Configurar la regla de protección en GitHub:
# 1. Ve a Settings > Branches > Add rule > Pattern: main.
# 2. Marca: 'Require a pull request before merging'.
# 3. Marca: 'Require approvals' (Mínimo: 1).
# 4. Marca: 'Do not allow bypassing the above settings'.
# 5. Pulsa 'Create'.

# PASO 3: Intentar hacer un push directo a main en la terminal (DEBE SER RECHAZADO)
echo "intento no autorizado" >> README.md
git commit -am "hack: push directo a main sin autorizacion"
git push origin main
# Salida esperada de GitHub:
# remote: error: GH006: Protected branch update failed for refs/heads/main.
# remote: Changes must be made through a pull request.

# PASO 4: Seguir el flujo corporativo correcto abriendo una rama
git switch -c fix/actualizar-documentacion
git push -u origin fix/actualizar-documentacion

# PASO 5: En GitHub, abrir el Pull Request:
# 1. Asignar título: fix: actualizar documentación del README.
# 2. Solicitar revisión.
# 3. Observar cómo el botón verde 'Merge pull request' aparece BLOQUEADO hasta recibir la aprobación.
# 4. Una vez aprobado el veredicto, seleccionar 'Squash and merge' y confirmar.
```

Conclusión de Calidad: Has implementado con éxito una compuerta de seguridad industrial (*Quality Gate*). La rama principal de producción ha quedado protegida contra errores humanos, modificaciones accidentales o ataques de inyección de código.

HOJA DE REFERENCIA RÁPIDA (CHEAT SHEET PROFESIONAL DE GIT)

REFERENCIA RÁPIDA

TEXT: GUÍA DE COMANDOS ESENCIALES PARA EL DÍA A DÍA — Terminal Cheat Sheet

```
# --- CREACIÓN Y CONFIGURACIÓN ---
git init                                # Iniciar repositorio local
git clone <url>                         # Clonar repositorio remoto existente
git config --global user.name "Nombre"  # Configurar nombre de autor
git config --global user.email "email"  # Configurar correo de autor

# --- CICLO DE TRABAJO LOCAL ---
git status -s                           # Ver estado simplificado de archivos
git add <archivo>                       # Pasar archivo específico al Staging Area
git add .                               # Pasar todos los cambios modificados a Staging
git commit -m "tipo: mensaje"           # Confirmar cambios con mensaje semántico
git commit --amend                      # Modificar el último commit realizado

# --- INSPECCIÓN Y COMPARACIÓN ---
git log --oneline --graph --all         # Visualizar historial en árbol visual
git diff                                # Ver cambios sin preparar (Working vs Staging)
git diff --staged                       # Ver cambios preparados (Staging vs Head)
git show <hash>                         # Ver el contenido detallado de un commit

# --- RAMAS Y FUSIONES ---
git branch -a                           # Listar todas las ramas (locales y remotas)
git switch -c <nombre-rama>             # Crear y cambiar a una nueva rama
git switch <rama>                       # Cambiar a una rama existente
git merge <rama>                        # Fusionar la rama indicada en la rama actual
git merge --abort                       # Cancelar un proceso de merge con conflicto
git branch -d <rama>                    # Eliminar una rama local ya fusionada

# --- SINCRONIZACIÓN CON GITHUB ---
git remote add origin <url-ssh>         # Vincular repositorio remoto como origin
git push -u origin <rama>               # Subir rama local al remoto con tracking
git pull origin <rama>                  # Descargar y fusionar cambios remotos
git fetch origin                        # Descargar metadatos remotos sin modificar disco

# --- DESHACER Y RECUPERACIÓN ---
git restore <archivo>                   # Descartar cambios sin guardar en archivo local
git restore --staged <archivo>          # Sacar archivo de Staging sin perder cambios
git revert <hash>                       # Crear commit inverso seguro para ramas públicas
git stash / git stash pop               # Guardar en gaveta temporal / Recuperar
```

Matriz de Criterios de Calidad en Repositorios Onetel C.A.

Área de Auditoría	Estándar Requerido	Validación
Seguridad de Credenciales	Cero archivos `.env`, llaves privadas o contraseñas en el historial.	`.gitignore` estricto
Gobernanza de Ramas	Rama `main` protegida con mínimo 1 aprobación y CI pasando.	Branch Rulesets
Historial Limpio	Mensajes con Conventional Commits y uso de *Squash and merge*.	Code Review en PR